CN Mini Project Demo (Group 9)

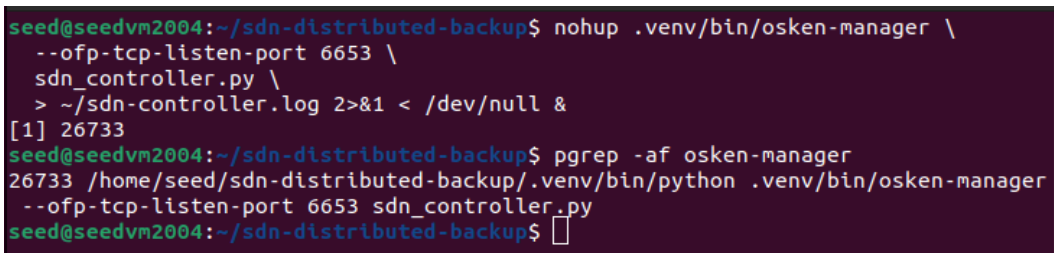
SDN-Based Distributed Backup System with Fast Failover

1.Start the controller
Terminal A:

cd ~/sdn-distributed-backup

```
nohup .venv/bin/osken-manager \
  --ofp-tcp-listen-port 6653 \
  sdn_controller.py \
  > ~/sdn-controller.log 2>&1 < /dev/null &

pgrep -af osken-manager
```



2.Start Mininet Topology
Terminal B:

cd ~/sdn-distributed-backup
sudo mn -c
sudo python3 sdn_topology.py

```
seed@seedvm2004:~/sdn-distributed-backup$ cd ~/sdn-distributed-backup
sudo mn -c
sudo python3 sdn_topology.py
[1]+  Terminated              nohup .venv/bin/osken-manager --ofp-tcp-listen-port
 6653 sdn_controller.py > ~/sdn-controller.log 2>&1 < /dev/null
*** Removing excess controllers/ofprotocols/ofdatapaths/pings/noxes
killall controller ofprotocol ofdatapath ping nox_corelt-nox_core ovs-openflowd o
vs-controllerovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
killall -9 controller ofprotocol ofdatapath ping nox_corelt-nox_core ovs-openflow
d ovs-controllerovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
pkill -9 -f "sudo mnexec"
*** Removing junk from /tmp
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log
*** Removing old X11 tunnels
*** Removing excess kernel datapaths
ps ax | egrep -o 'dp[0-9]+' | sed 's/dp/nl:/'
***  Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethX
ip link show | egrep -o '([-_.[:alnum:]]+-eth[[:digit:]]+)'
ip link show
*** Killing stale mininet node processes
pkill -9 -f mininet:
*** Shutting down stale tunnels
pkill -9 -f Tunnel=Ethernet
pkill -9 -f .ssh/mn
rm -f ~/.ssh/mn/*
*** Cleanup complete.
*** Creating network
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1 s2
*** Adding links:
(h1, s1) (h2, s2) (h3, s2) (s1, s2) (s1, s2)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 2 switches
s1 s2 ...

Backup SDN topology started.
Hosts: h1 (client), h2 and h3 (backup nodes)
Try: pingall
*** Starting CLI:
mininet>
```

nodes
net
links

```
mininet> nodes
available nodes are:
c0 h1 h2 h3 s1 s2
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s2-eth3
h3 h3-eth0:s2-eth4
s1 lo:  s1-eth1:h1-eth0 s1-eth2:s2-eth1 s1-eth3:s2-eth2
s2 lo:  s2-eth1:s1-eth2 s2-eth2:s1-eth3 s2-eth3:h2-eth0 s2-eth4:h3-eth0
c0
mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s2-eth3 (OK OK)
h3-eth0<->s2-eth4 (OK OK)
s1-eth2<->s2-eth1 (OK OK)
s1-eth3<->s2-eth2 (OK OK)
mininet>
```

3.Test Network Connectivity

pingall

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
mininet>
```

4.Start both backup servers

h2 bash -c 'cd /home/seed/sdn-distributed-backup; export BACKUP_PORT=9001 BACKUP_DIR=/tmp/backup-h2; nohup python3 -u server.py >/tmp/backup-h2.log 2>&1 </dev/null &'

h3 bash -c 'cd /home/seed/sdn-distributed-backup; export BACKUP_PORT=9002 BACKUP_DIR=/tmp/backup-h3; nohup python3 -u server.py >/tmp/backup-h3.log 2>&1 </dev/null &'

sh cat /tmp/backup-h2.log

sh cat /tmp/backup-h3.log

```
mininet> h2 bash -c 'cd /home/seed/sdn-distributed-backup; export BACKUP_PORT=900
1 BACKUP_DIR=/tmp/backup-h2; nohup python3 -u server.py >/tmp/backup-h2.log 2>&1
</dev/null &'
mininet> h3 bash -c 'cd /home/seed/sdn-distributed-backup; export BACKUP_PORT=900
2 BACKUP_DIR=/tmp/backup-h3; nohup python3 -u server.py >/tmp/backup-h3.log 2>&1
</dev/null &'
mininet> sh cat /tmp/backup-h2.log
Backup server listening on 0.0.0.0:9001
Files will be saved in: /tmp/backup-h2
Partial uploads and manifests: /tmp/backup-h2/.partial
mininet> sh cat /tmp/backup-h3.log
Backup server listening on 0.0.0.0:9002
Files will be saved in: /tmp/backup-h3
Partial uploads and manifests: /tmp/backup-h3/.partial
mininet>
```

5.Create test file

h1 dd if=/dev/urandom of=/tmp/demo-backup.bin bs=1M count=5 status=none
h1 ls -lh /tmp/demo-backup.bin
h1 sha256sum /tmp/demo-backup.bin

(create test file ->check size -> calculate the file's SHA-256 hash)

```
mininet> h1 dd if=/dev/urandom of=/tmp/demo-backup.bin bs=1M count=5 status=none
mininet> h1 ls -lh /tmp/demo-backup.bin
-rw-r--r-- 1 root root 5.0M Oct 21 11:50 /tmp/demo-backup.bin
mininet> h1 sha256sum /tmp/demo-backup.bin
11cd6c0c880765dcc0234e8df1ff854f7e848ccfc186b912161baaaf7f46258e  /tmp/demo-backu
p.bin
mininet>
```

5.Upload test file to both backup servers

h1 python3 /home/seed/sdn-distributed-backup/distributed_client.py /tmp/demo-backup.bin
10.0.0.2:9001 10.0.0.3:9002

```
mininet> h1 python3 /home/seed/sdn-distributed-backup/distributed_client.py /tmp/
demo-backup.bin 10.0.0.2:9001 10.0.0.3:9002

=== Replicating to 10.0.0.2:9001 ===
File: demo-backup.bin
Size: 5242880 bytes
SHA-256: 11cd6c0c880765dcc0234e8df1ff854f7e848ccfc186b912161baaaf7f46258e
Transferred 5242880/5242880 bytes (100.0%)
Upload complete: demo-backup.bin
Elapsed: 0.26 seconds
Throughput: 19.01 MiB/s

=== Replicating to 10.0.0.3:9002 ===
File: demo-backup.bin
Size: 5242880 bytes
SHA-256: 11cd6c0c880765dcc0234e8df1ff854f7e848ccfc186b912161baaaf7f46258e
Transferred 5242880/5242880 bytes (100.0%)
Upload complete: demo-backup.bin
Elapsed: 0.07 seconds
Throughput: 70.66 MiB/s

=== DISTRIBUTED BACKUP SUMMARY ===
Successful nodes: 2/2
  OK: 10.0.0.2:9001
  OK: 10.0.0.3:9002
mininet>
```

6.Verify that both copies match the original (checksum verification)

sh ls -lh /tmp/backup-h2/demo-backup.bin /tmp/backup-h3/demo-backup.bin
sh sha256sum /tmp/demo-backup.bin /tmp/backup-h2/demo-backup.bin
/tmp/backup-h3/demo-backup.bin

```
mininet> sh ls -lh /tmp/backup-h2/demo-backup.bin /tmp/backup-h3/demo-backup.bin
-rw-r--r-- 1 root root 5.0M Oct 21 11:51 /tmp/backup-h2/demo-backup.bin
-rw-r--r-- 1 root root 5.0M Oct 21 11:51 /tmp/backup-h3/demo-backup.bin
mininet> sh sha256sum /tmp/demo-backup.bin /tmp/backup-h2/demo-backup.bin /tmp/bac
kup-h3/demo-backup.bin
11cd6c0c880765dcc0234e8df1ff854f7e848ccfc186b912161baaaf7f46258e  /tmp/demo-backup
.bin
11cd6c0c880765dcc0234e8df1ff854f7e848ccfc186b912161baaaf7f46258e  /tmp/backup-h2/d
emo-backup.bin
11cd6c0c880765dcc0234e8df1ff854f7e848ccfc186b912161baaaf7f46258e  /tmp/backup-h3/d
emo-backup.bin
mininet>
```

7.Pytest

Integration tests

Test	Purpose	Result
`test_upload_and_sha256_verification`	Upload a file and compare the stored file's SHA-256 digest with the source	PASS
`test_replication_to_two_servers`	Upload to two independent backup servers and verify both copies	PASS
`test_interrupted_upload_resumes`	Interrupt a transfer after a chunk acknowledgement and resume it	PASS
`test_corrupted_payload_is_rejected`	Send bytes that do not match the declared digest and check that finalization is rejected	PASS

Protocol unit tests

Test	Purpose	Result
`test_fragmented_message`	Handle a message arriving in fragments	PASS
`test_invalid_header_length`	Reject an invalid message length	PASS
`test_invalid_json`	Reject malformed JSON	PASS
`test_json_round_trip`	Verify JSON message encoding and decoding	PASS
`test_recv_exact_incomplete_data`	Handle incomplete data when reading an exact byte count	PASS

cd ~/sdn-distributed-backup
.venv/bin/python -m pytest -v

```
seed@seedvm2004:~/sdn-distributed-backup$ cd ~/sdn-distributed-backup
.venv/bin/python -m pytest -v
============================= test session starts ==============================
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.7.0 -- /home/seed/sdn-d
istributed-backup/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/seed/sdn-distributed-backup
configfile: pytest.ini
testpaths: tests
collected 9 items

tests/test_integration.py::test_upload_and_sha256_verification PASSED    [ 11%]
tests/test_integration.py::test_replication_to_two_servers PASSED        [ 22%]
tests/test_integration.py::test_interrupted_upload_resumes PASSED        [ 33%]
tests/test_integration.py::test_corrupted_payload_is_rejected PASSED     [ 44%]
tests/test_protocol.py::TestProtocol::test_fragmented_message PASSED     [ 55%]
tests/test_protocol.py::TestProtocol::test_invalid_header_length PASSED  [ 66%]
tests/test_protocol.py::TestProtocol::test_invalid_json PASSED           [ 77%]
tests/test_protocol.py::TestProtocol::test_json_round_trip PASSED        [ 88%]
tests/test_protocol.py::TestProtocol::test_recv_exact_incomplete_data PASSED [10
0%]

============================== 9 passed in 0.71s ===============================
seed@seedvm2004:~/sdn-distributed-backup$
```

8.SDN Failover Demo

a)Ping before failure

h1 ping -c 3 10.0.0.2
h1 ping -c 3 10.0.0.3

```
mininet> h1 ping -c 3 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=2.94 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.082 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.078 ms

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 0.078/1.032/2.936/1.346 ms
mininet> h1 ping -c 3 10.0.0.3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=1.76 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=0.103 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=0.686 ms

--- 10.0.0.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2027ms
rtt min/avg/max/mdev = 0.103/0.849/1.760/0.686 ms
```

b)Failure

sh ip link set dev s1-eth2 down

```
mininet> sh ip link set dev s1-eth2 down
```

c)Ping after failure

h1 ping -c 5 10.0.0.2
h1 ping -c 5 10.0.0.3

```
mininet> h1 ping -c 5 10.0.0.3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=6.89 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=0.367 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=0.115 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=0.302 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=0.203 ms

--- 10.0.0.3 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4056ms
rtt min/avg/max/mdev = 0.115/1.575/6.888/2.657 ms
mininet> h1 ping -c 5 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.760 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.163 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.289 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.286 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.144 ms

--- 10.0.0.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4078ms
rtt min/avg/max/mdev = 0.144/0.328/0.760/0.224 ms
```

d)Test with actual upload, verify hashes

h1 dd if=/dev/urandom of=/tmp/failover-demo.bin bs=1M count=5 status=none
h1 python3 /home/seed/sdn-distributed-backup/distributed_client.py /tmp/failover-demo.bin 10.0.0.2:9001 10.0.0.3:9002

sh sha256sum /tmp/failover-demo.bin /tmp/backup-h2/failover-demo.bin /tmp/backup-h3/failover-demo.bin

```
mininet> h1 dd if=/dev/urandom of=/tmp/failover-demo.bin bs=1M count=5 status=none
mininet> h1 python3 /home/seed/sdn-distributed-backup/distributed_client.py /tmp/f
ailover-demo.bin 10.0.0.2:9001 10.0.0.3:9002

=== Replicating to 10.0.0.2:9001 ===
File: failover-demo.bin
Size: 5242880 bytes
SHA-256: a79548d0b6ef77a98638e6bb073812898f1401d4541e3c8910b539b57b9f7746
Transferred 5242880/5242880 bytes (100.0%)
Upload complete: failover-demo.bin
Elapsed: 0.09 seconds
Throughput: 57.61 MiB/s

=== Replicating to 10.0.0.3:9002 ===
File: failover-demo.bin
Size: 5242880 bytes
SHA-256: a79548d0b6ef77a98638e6bb073812898f1401d4541e3c8910b539b57b9f7746
Transferred 5242880/5242880 bytes (100.0%)
Upload complete: failover-demo.bin
Elapsed: 0.10 seconds
Throughput: 52.16 MiB/s

=== DISTRIBUTED BACKUP SUMMARY ===
Successful nodes: 2/2
  OK: 10.0.0.2:9001
  OK: 10.0.0.3:9002
mininet> sh sha256sum /tmp/failover-demo.bin /tmp/backup-h2/failover-demo.bin /tmp
/backup-h3/failover-demo.bin
a79548d0b6ef77a98638e6bb073812898f1401d4541e3c8910b539b57b9f7746  /tmp/failover-de
mo.bin
a79548d0b6ef77a98638e6bb073812898f1401d4541e3c8910b539b57b9f7746  /tmp/backup-h2/f
ailover-demo.bin
a79548d0b6ef77a98638e6bb073812898f1401d4541e3c8910b539b57b9f7746  /tmp/backup-h3/f
ailover-demo.bin
mininet>
```

(sh ip link set dev s1-eth2 up
pingall)

THEORY:

- **Topology :**



Function of each element :

Device	Purpose
h1	Runs the client that sends the file for backup.
s1	Connects the client to the rest of the network.
s2	Connects the network to both backup servers.
h2	Receives and stores the first backup copy.
h3	Receives and stores the second backup copy.
Controller	Programs the switches' forwarding and failover rules.

- **Mininet : It lets you create a virtual network containing hosts, switches, and links on one Linux machine.**
- **Open vSwitch provides the virtual switches**
- **OS-Ken runs the SDN controller**

Your benchmark data shows how the time and throughput of your distributed backup system change as the file size increases.

In your project, the benchmark script tests files of 1, 5, 10, and 25 MiB, with three trials per file size. Each file is replicated to both backup servers (`10.0.0.2:9001` and `10.0.0.3:9002`), and successful completion and SHA-256 integrity are verified.

What the benchmark measured

- **File size: How much data is being backed up.**
- **Elapsed time: How long the distributed upload takes to complete.**
- **Logical throughput: Original file size divided by elapsed time, in MiB/s.**
- **Aggregate replicated throughput: Throughput accounting for the data sent to both backup servers.**

Results

The recorded trial results were:

File size	Trial times (seconds)	Logical throughput (MiB/s)
1 MiB	0.261, 0.073, 0.053	3.83, 13.79, 18.93
5 MiB	0.249, 0.131, 0.244	20.07, 38.07, 20.54
10 MiB	0.221, 0.242, 0.146	45.28, 41.32, 68.45
25 MiB	0.633, 0.398, 0.356	39.52, 62.85, 70.30

These are the recorded individual trials, not theoretical or maximum network speeds.

What the graphs tell you

Throughput generally improves with larger files.

The 1 MiB uploads achieved 3.83–18.93 MiB/s, while the 10 and 25 MiB uploads reached as high as 68.45 and 70.30 MiB/s respectively. This suggests that fixed overheads have a larger impact on small uploads.

Larger files generally take longer, but not proportionally.

The 25 MiB uploads took 0.356–0.633 seconds, compared with 0.053–0.261 seconds for 1 MiB. Individual timings vary between trials.

Performance measurements were accompanied by correctness checks.

The benchmark trials completed successfully, with file integrity verified using SHA-256. That matters because a fast transfer is not useful if the backup is incomplete or corrupted.